



模式识别与机器学习 (11)

多层感知器算法-2

左旺孟

综合楼712

视觉感知与认知组

哈尔滨工业大学计算机学院

cswmzuo@gmail.com

13134506692

主要内容

- 随机优化算法（Stochastic Optimization）
 - 随机优化
 - SGD、AdaGrad、RMSProp、ADAM、
 - ADAMW、LAMB、SAM
- 网络模型：3层 -> 多层
 - 激活函数：ReLU / PReLU, ...
 - 网络参数：Dropout
- 泛化能力

常见的损失函数

- 回归问题
 - 均方差损失

$$f^* = \arg \min_f \mathbb{E}_{\mathbf{x}, \mathbf{y} \sim p_{\text{data}}} \|\mathbf{y} - f(\mathbf{x})\|^2$$

- L1损失

$$f^* = \arg \min_f \mathbb{E}_{\mathbf{x}, \mathbf{y} \sim p_{\text{data}}} \|\mathbf{y} - f(\mathbf{x})\|_1$$

常见的损失函数

- 分类问题
 - 两类问题

$$z = \mathbf{w}^\top \mathbf{h} + b \quad f(x) = \frac{1}{1 + e^{-x}}$$

$$\begin{aligned} J(\boldsymbol{\theta}) &= -\log P(y \mid \mathbf{x}) \\ &= -\log \sigma((2y - 1)z) \\ &= \zeta((1 - 2y)z). \end{aligned}$$

- 多类问题

$$\mathbf{z} = \mathbf{W}^\top \mathbf{h} + \mathbf{b} \quad \text{softmax}(\mathbf{z})_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

$$z_i - \log \sum_j \exp(z_j)$$

随机优化 (Stochastic Optimization)

- 随机优化 & SGD

$$J(\boldsymbol{\theta}) = \mathbb{E}_{\mathbf{x}, y \sim \hat{p}_{\text{data}}} L(\mathbf{x}, y, \boldsymbol{\theta}) = \frac{1}{m} \sum_{i=1}^m L(\mathbf{x}^{(i)}, y^{(i)}, \boldsymbol{\theta})$$

- $L(\mathbf{x}, y) = \frac{1}{2} \|\mathbf{x} - y\|^2$

- 梯度下降

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \frac{1}{m} \sum_{i=1}^m \nabla_{\boldsymbol{\theta}} L(\mathbf{x}^{(i)}, y^{(i)}, \boldsymbol{\theta})$$

- 随机梯度下降 (SGD) (小批量)

$$g = \frac{1}{m'} \nabla_{\boldsymbol{\theta}} \sum_{i=1}^{m'} L(\mathbf{x}^{(i)}, y^{(i)}, \boldsymbol{\theta}) \quad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \epsilon g,$$

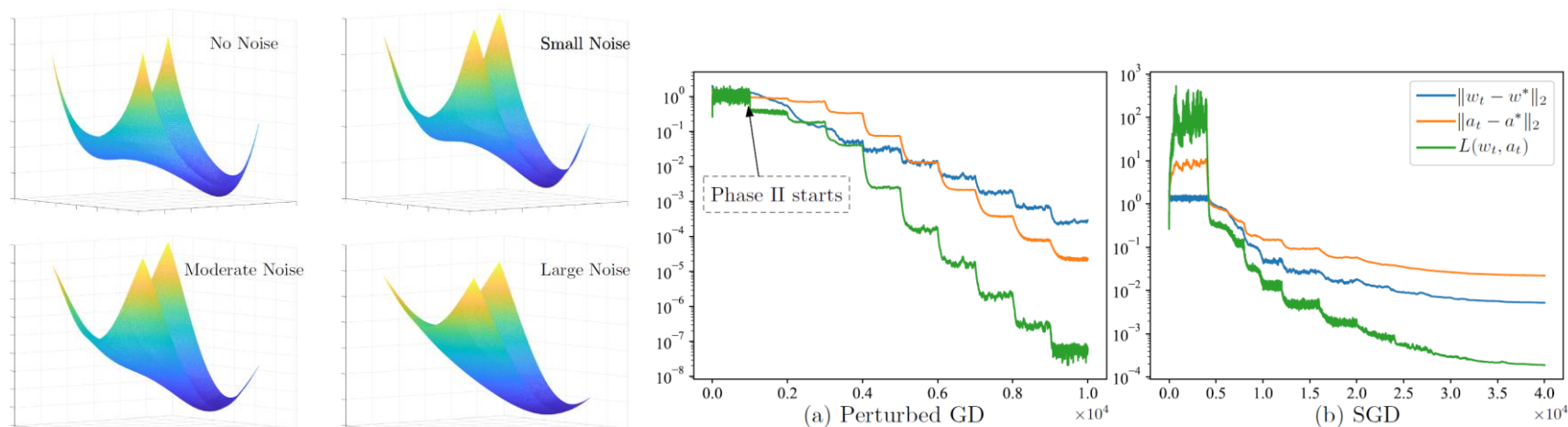


随机优化 (Stochastic Optimization)

- 化整为零
- 有助于逃离一部分局部最优解

深度神经网络的泛化能力

- 目前的学习算法理论上只能收敛到一个局部最优解。
- 实际看来，借助于学习率的调整，学习到的模型很大概率能够绕过一些不好的局部最优解（SGD）



Gradient descent finds global minima of deep neural networks, arXiv:1811.03804;

Towards Understanding the Importance of Noise in Training Neural Networks,
ICML 2019

随机优化

- 最大均值差异 (Maximum Mean Discrepancy, MMD)

$$\text{MMD}^2(s, t) = \sup_{\|\phi\|_{\mathcal{H}} \leq 1} \left\| E_{\mathbf{x}^s \sim s} [\phi(\mathbf{x}^s)] - E_{\mathbf{x}^t \sim t} [\phi(\mathbf{x}^t)] \right\|_{\mathcal{H}}^2$$

$$\text{MMD}^2(\mathcal{D}_s, \mathcal{D}_t) = \left\| \frac{1}{M} \sum_{i=1}^M \phi(\mathbf{x}_i^s) - \frac{1}{N} \sum_{j=1}^N \phi(\mathbf{x}_j^t) \right\|_{\mathcal{H}}^2$$

A. Gretton, K. Borgwardt, M. Rasch, B. Scholkopf, and A. Smola. A kernel method for the two-sample problem. In Advances in Neural Information Processing Systems 15, pages 513–520, Cambridge, MA, 2007a. MIT Press.

随机优化

$$\begin{aligned}\text{MMD}^2[\mathcal{F}, p, q] &:= \langle \mu_p - \mu_q, \mu_p - \mu_q \rangle_{\mathcal{H}} \\ &= \langle \mu_p, \mu_p \rangle_{\mathcal{H}} + \langle \mu_q, \mu_q \rangle_{\mathcal{H}} - 2\langle \mu_p, \mu_q \rangle_{\mathcal{H}} \\ &= \mathbf{E}_p \langle \phi(x), \phi(x') \rangle_{\mathcal{H}} + \mathbf{E}_q \langle \phi(y), \phi(y') \rangle_{\mathcal{H}} \\ &\quad - 2\mathbf{E}_{p, q} \langle \phi(x), \phi(y) \rangle_{\mathcal{H}},\end{aligned}$$

- Mini-batch计算（4元组Batch） $\{\mathbf{x}_i, \mathbf{x}'_i; \mathbf{y}_i, \mathbf{y}'_i\}$

$$g = \frac{1}{m'} \nabla_{\theta} \sum_{i=1}^{m'} L(\mathbf{x}^{(i)}, \mathbf{y}^{(i)}, \theta) \Rightarrow \frac{1}{m'} \sum_{i=1}^{m'} \frac{\partial \langle \phi(\mathbf{x}_i), \phi(\mathbf{x}'_i) \rangle}{\partial \theta} + \frac{\partial \langle \phi(\mathbf{y}_i), \phi(\mathbf{y}'_i) \rangle}{\partial \theta} - 2 \frac{\partial \langle \phi(\mathbf{x}_i), \phi(\mathbf{y}_i) \rangle}{\partial \theta}$$

随机优化

- 随机ADMM (Alternating Direction Method of Multipliers)

$$\min_{\mathbf{x} \in \mathcal{X}, \mathbf{y} \in \mathcal{Y}} \mathbb{E}_{\boldsymbol{\xi}} \theta_1(\mathbf{x}, \boldsymbol{\xi}) + \theta_2(\mathbf{y}) \text{ s.t. } A\mathbf{x} + B\mathbf{y} = \mathbf{b},$$

$$\min_{\mathbf{x} \in \mathcal{X}, \mathbf{y} \in \mathcal{Y}} \mathcal{L}_{\beta}(\mathbf{x}, \mathbf{y}, \boldsymbol{\lambda}) \equiv \min_{\mathbf{x} \in \mathcal{X}, \mathbf{y} \in \mathcal{Y}} \left[\theta_1(\mathbf{x}) + \theta_2(\mathbf{y}) - \langle \boldsymbol{\lambda}, A\mathbf{x} + B\mathbf{y} - \mathbf{b} \rangle + \frac{\beta}{2} \|A\mathbf{x} + B\mathbf{y} - \mathbf{b}\|^2 \right], \quad \theta_1(\mathbf{x}, \boldsymbol{\xi}_{k+1})$$

$$\hat{\mathcal{L}}_{\beta,k}(\mathbf{x}, \mathbf{y}, \boldsymbol{\lambda}) \equiv \theta_1(\mathbf{x}_k) + \langle \theta'_1(\mathbf{x}_k, \boldsymbol{\xi}_{k+1}), \mathbf{x} \rangle + \theta_2(\mathbf{y}) - \langle \boldsymbol{\lambda}, A\mathbf{x} + B\mathbf{y} - \mathbf{b} \rangle + \frac{\beta}{2} \|A\mathbf{x} + B\mathbf{y} - \mathbf{b}\|^2 + \frac{\|\mathbf{x} - \mathbf{x}_k\|^2}{2\eta_{k+1}}$$



随机优化

- Batch
 - 样本Batch、样本对Batch、（三）四元组Batch
- 学习算法
 - SGD（当前深度学习中的主流方法）
 - 其他优化算法如S-ADMM（可否用于深度网络训练？）

Training Neural Networks Without Gradients: A Scalable ADMM Approach,
ICML 2016



随机梯度下降 (SGD)

算法 8.1 随机梯度下降 (SGD) 在第 k 个训练迭代的更新

Require: 学习率 ϵ_k

Require: 初始参数 θ

while 停止准则未满足 **do**

从训练集中采包含 m 个样本 $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ 的小批量, 其中 $\mathbf{x}^{(i)}$ 对应目标为 $\mathbf{y}^{(i)}$ 。

计算梯度估计: $\hat{\mathbf{g}} \leftarrow +\frac{1}{m} \nabla_{\theta} \sum_i L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$

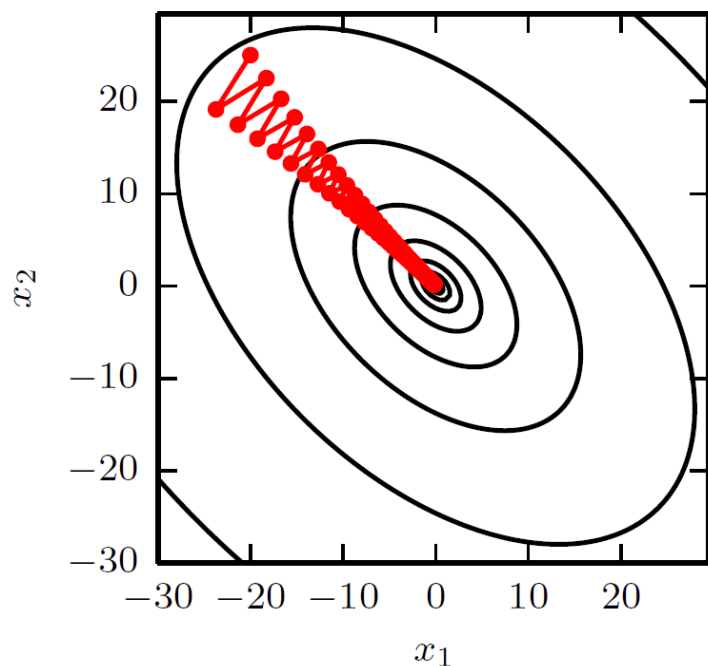
应用更新: $\theta \leftarrow \theta - \epsilon \hat{\mathbf{g}}$

end while

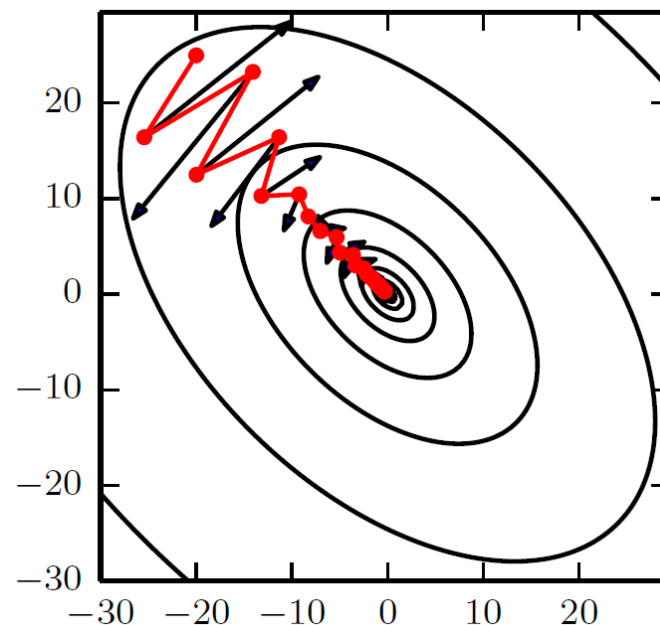
- 学习率: $\sum_{k=1}^{\infty} \epsilon_k = \infty, \quad \sum_{k=1}^{\infty} \epsilon_k^2 < \infty$
- 小批量

动量方法（SGD-Momentum）

- 梯度下降的局限性：病态条件的二次目标函数



SGD



SGD-Momentum

动量方法（SGD-Momentum）

- 动量（物理）：质量 x 速度
- 神经网络学习
 - 单位质量
 - 速度 $\mathbf{v}$

$$\mathbf{v} \leftarrow \alpha \mathbf{v} - \epsilon \nabla_{\theta} \left(\frac{1}{m} \sum_{i=1}^m L(\mathbf{f}(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)}) \right)$$

- α : 一般取值为0.5, 0.9 和0.99

Nesterov加速梯度

- 凸优化问题:

$$\mathbf{x}_k = \mathbf{y}_{k-1} - \varepsilon \nabla F(\mathbf{y}_{k-1}) \quad (\varepsilon \leq 1/L_F)$$

$$t_{k+1} \leftarrow (1 + \sqrt{1 + 4t_k^2})/2,$$

$$\mathbf{y}_{k+1} \leftarrow \mathbf{x}_k + (t_k - 1)/t_{k+1}(\mathbf{x}_k - \mathbf{x}_{k-1})$$

Theorem 1: Let $\{\mathbf{x}_k\}$ be generated by the APG method and $\mathbf{x}^*$ be any optimal solution, then

$$F(\mathbf{x}_k) - F(\mathbf{x}^*) \leq \frac{2L_f \|\mathbf{x}_0 - \mathbf{x}^*\|_F^2}{(k+1)^2}, \quad \forall k \geq 1. \quad (20)$$

A Generalized Accelerated Proximal Gradient Approach for Total-Variation-Based Image, IEEE T-IP, 2010.

Nesterov 动量

$$\begin{aligned} \mathbf{v} &\leftarrow \alpha \mathbf{v} - \epsilon \nabla_{\boldsymbol{\theta}} \left[\frac{1}{m} \sum_{i=1}^m L(f(\mathbf{x}^{(i)}; \boldsymbol{\theta} + \alpha \mathbf{v}), \mathbf{y}^{(i)}) \right] \\ \boldsymbol{\theta} &\leftarrow \boldsymbol{\theta} + \mathbf{v}, \end{aligned}$$

算法 8.3 使用 Nesterov 动量的随机梯度下降 (SGD)

Require: 学习率 ϵ , 动量参数 α

Require: 初始参数 $\boldsymbol{\theta}$, 初始速度 $\mathbf{v}$

while 没有达到停止准则 **do**

从训练集中采包含 m 个样本 $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ 的小批量, 对应目标为 $\mathbf{y}^{(i)}$ 。

应用临时更新: $\tilde{\boldsymbol{\theta}} \leftarrow \boldsymbol{\theta} + \alpha \mathbf{v}$

计算梯度 (在临时点): $\mathbf{g} \leftarrow \frac{1}{m} \nabla_{\tilde{\boldsymbol{\theta}}} \sum_i L(f(\mathbf{x}^{(i)}; \tilde{\boldsymbol{\theta}}), \mathbf{y}^{(i)})$

计算速度更新: $\mathbf{v} \leftarrow \alpha \mathbf{v} - \epsilon \mathbf{g}$

应用更新: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \mathbf{v}$

end while

在随机梯度的情况下, Nesterov动量没有改进收敛率。

AdaGrad

- 具有损失最大偏导的参数相应地有一个快速下降的学习率。在参数空间中更为平缓的倾斜方向会取得更大的下降。

算法 8.4 AdaGrad 算法

Require: 全局学习率 ϵ

Require: 初始参数 θ

Require: 小常数 δ , 为了数值稳定大约设为 10^{-7}

初始化梯度累积变量 $r = 0$

while 没有达到停止准则 **do**

从训练集中采包含 m 个样本 $\{x^{(1)}, \dots, x^{(m)}\}$ 的小批量, 对应目标为 $y^{(i)}$ 。

计算梯度: $g \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(x^{(i)}; \theta), y^{(i)})$

累积平方梯度: $r \leftarrow r + g \odot g$

计算更新: $\Delta\theta \leftarrow -\frac{\epsilon}{\delta + \sqrt{r}} \odot g$ (逐元素地应用除和求平方根)

应用更新: $\theta \leftarrow \theta + \Delta\theta$

end while

AdaGrad

- 凸优化情况下有较好的理论性质，但对于深度神经网络一般不成立
- 对于训练深度神经网络模型，从训练开始时积累梯度平方会导致有效学习率在达到这样的凸结构前过早和过量的减小。
- 在某些深度学习模型上效果不错，但不是全部。

RMSProp

- 使用指数衰减平均以丢弃早期历史，使其能够在找到凸结构后快速收敛；
- 相比于AdaGrad，基于移动平均引入了一个新的超参数 ρ ，用于控制移动平均的长度范围。
- 可结合SGD和Nesterov 动量方法

RMSProp

算法 8.5 RMSProp 算法

Require: 全局学习率 ϵ , 衰减速率 ρ

Require: 初始参数 θ

Require: 小常数 δ , 通常设为 10^{-6} (用于被小数除时的数值稳定)

初始化累积变量 $r = 0$

while 没有达到停止准则 **do**

从训练集中采包含 m 个样本 $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ 的小批量, 对应目标为 $\mathbf{y}^{(i)}$ 。

计算梯度: $\mathbf{g} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$

累积平方梯度: $\mathbf{r} \leftarrow \rho \mathbf{r} + (1 - \rho) \mathbf{g} \odot \mathbf{g}$

计算参数更新: $\Delta \theta = -\frac{\epsilon}{\sqrt{\delta + \mathbf{r}}} \odot \mathbf{g}$ ($\frac{1}{\sqrt{\delta + \mathbf{r}}}$ 逐元素应用)

应用更新: $\theta \leftarrow \theta + \Delta \theta$

end while

RMSPprop-2

算法 8.6 使用 Nesterov 动量的 RMSPprop 算法

Require: 全局学习率 ϵ , 衰减速率 ρ , 动量系数 α

Require: 初始参数 θ , 初始参数 v

初始化累积变量 $r = 0$

while 没有达到停止准则 **do**

从训练集中采包含 m 个样本 $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ 的小批量, 对应目标为 $\mathbf{y}^{(i)}$ 。

计算临时更新: $\tilde{\theta} \leftarrow \theta + \alpha v$

计算梯度: $\mathbf{g} \leftarrow \frac{1}{m} \nabla_{\tilde{\theta}} \sum_i L(f(\mathbf{x}^{(i)}; \tilde{\theta}), \mathbf{y}^{(i)})$

累积梯度: $\mathbf{r} \leftarrow \rho \mathbf{r} + (1 - \rho) \mathbf{g} \odot \mathbf{g}$

计算速度更新: $\mathbf{v} \leftarrow \alpha \mathbf{v} + \frac{\epsilon}{\sqrt{\mathbf{r}}} \odot \mathbf{g}$ ($\frac{1}{\sqrt{r}}$ 逐元素应用)

应用更新: $\theta \leftarrow \theta + \mathbf{v}$

end while

ADAM: ADaptive Moments

算法 8.7 Adam 算法

Require: 步长 ϵ (建议默认为: 0.001)

Require: 矩估计的指数衰减速率, ρ_1 和 ρ_2 在区间 $[0, 1)$ 内。(建议默认为: 分别为 0.9 和 0.999)

Require: 用于数值稳定的小常数 δ (建议默认为: 10^{-8})

Require: 初始参数 θ

初始化一阶和二阶矩变量 $s = 0, r = 0$

初始化时间步 $t = 0$

while 没有达到停止准则 **do**

从训练集中采包含 m 个样本 $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ 的小批量, 对应目标为 $\mathbf{y}^{(i)}$ 。

计算梯度: $\mathbf{g} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$

$t \leftarrow t + 1$

更新有偏一阶矩估计: $\mathbf{s} \leftarrow \rho_1 \mathbf{s} + (1 - \rho_1) \mathbf{g}$

更新有偏二阶矩估计: $\mathbf{r} \leftarrow \rho_2 \mathbf{r} + (1 - \rho_2) \mathbf{g} \odot \mathbf{g}$

修正一阶矩的偏差: $\hat{\mathbf{s}} \leftarrow \frac{\mathbf{s}}{1 - \rho_1^t}$

修正二阶矩的偏差: $\hat{\mathbf{r}} \leftarrow \frac{\mathbf{r}}{1 - \rho_2^t}$

计算更新: $\Delta \theta = -\epsilon \frac{\hat{\mathbf{s}}}{\sqrt{\hat{\mathbf{r}} + \delta}}$ (逐元素应用操作)

应用更新: $\theta \leftarrow \theta + \Delta \theta$

end while

- ADAM中动量直接并入了梯度一阶矩（指数加权）的估计。
- ADAM 包括了偏置修正，修正从原点初始化的一阶矩（动量项）和（非中心的）二阶矩的估计。
- Adam 通常被认为对超参数的选择相当鲁棒，尽管学习率有时需要修改建议的默认值。

小结

- 基本算法
 - SGD、SGD-Momentum、Nesterov 动量
- 自适应学习率算法
 - AdaGrad、RMSProp、ADAM
- 二阶及其近似：牛顿法、共轭梯度、BFGS
- 建议使用
 - SGD、SGD-Momentum、RMSProp、ADAM

最新进展

- **ADAMW**

Decoupled Weight Decay Regularization, ICLR 2019.

- **LAMB**

Large Batch Optimization for Deep Learning: Training BERT in 76 minutes, ICLR 2020.

- **SAM**

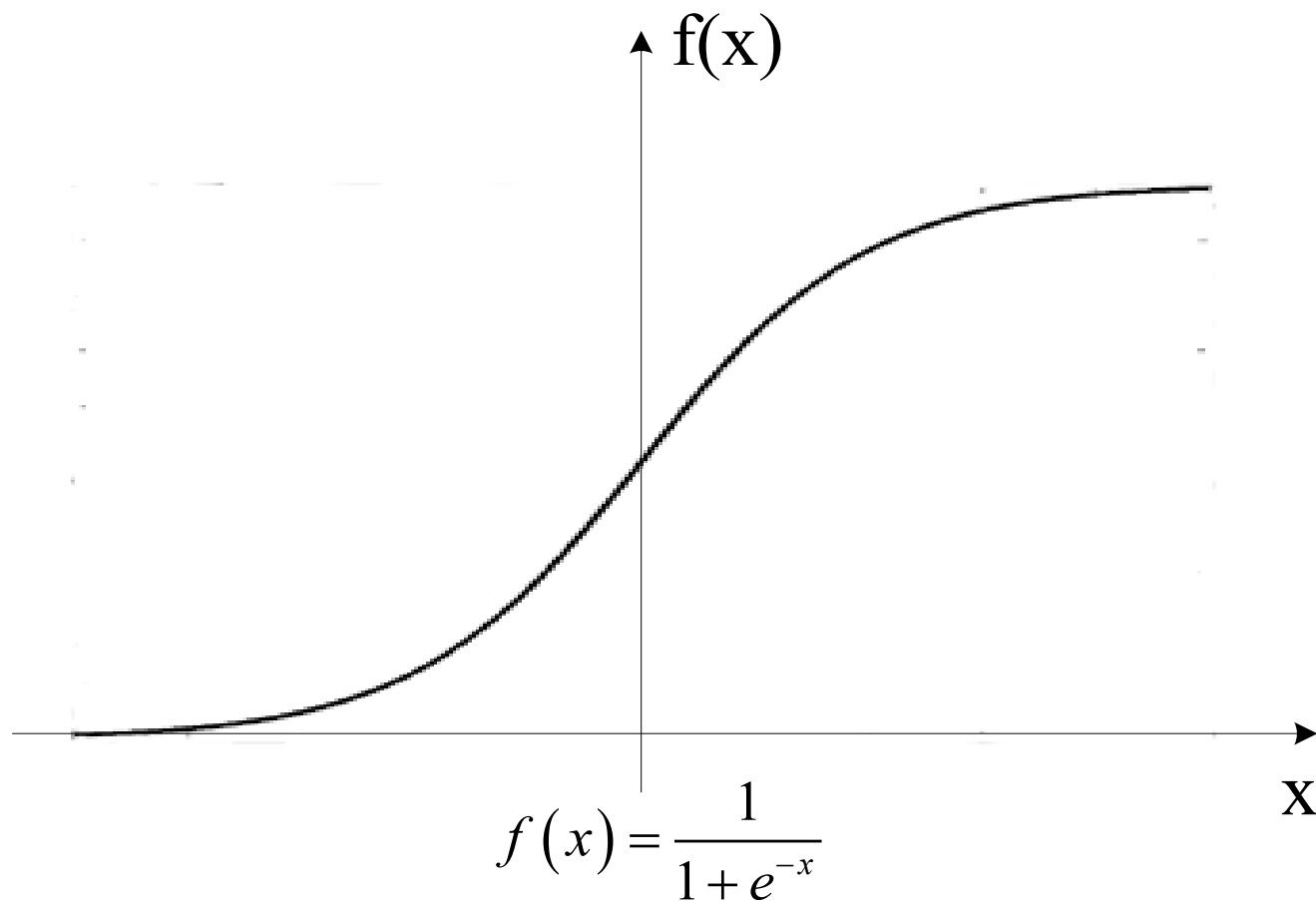
Sharpness-Aware Minimization for Efficiently Improving Generalization, ICLR 2021.

- ...

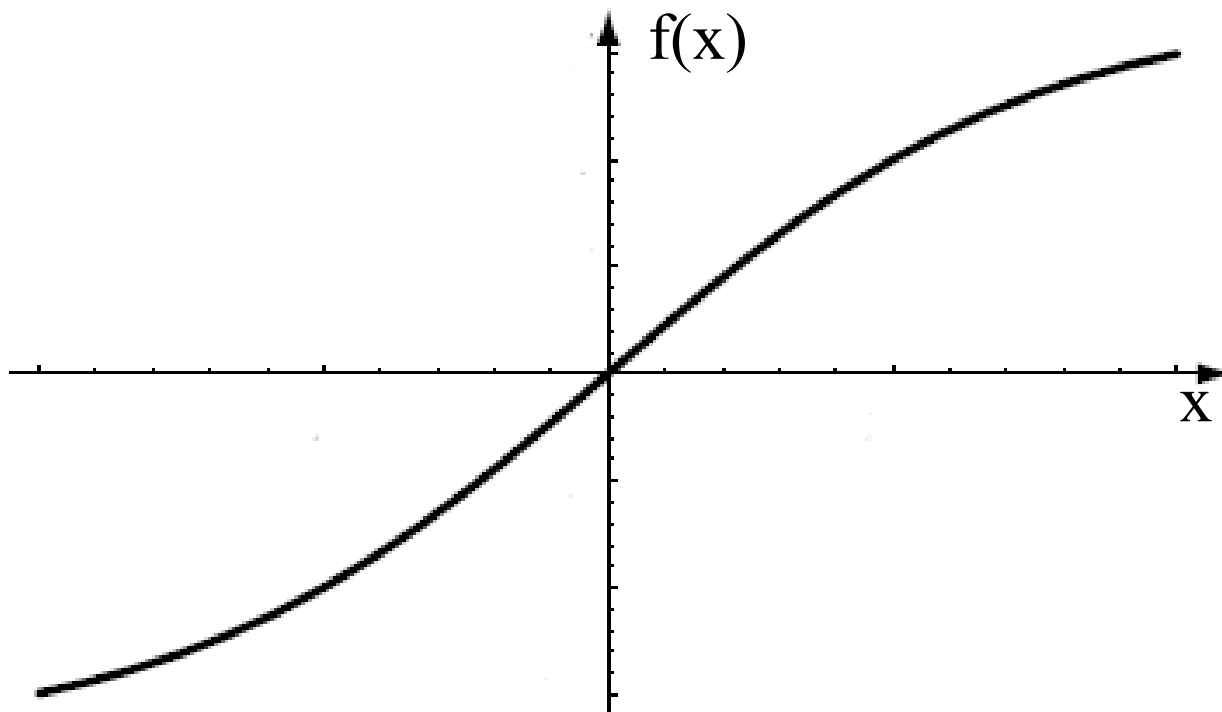
主要内容

- 随机优化算法（Stochastic Optimization）
 - 随机优化
 - SGD、AdaGrad、RMSProp、ADAM
- 网络模型：3层 -> 多层
 - 激活函数：ReLU / PReLU, ...
 - 网络学习：Dropout、批标准化（Batch Normalization）
- 泛化能力

激活函数——对数Sigmoid函数



激活函数—双曲正切Sigmoid函数

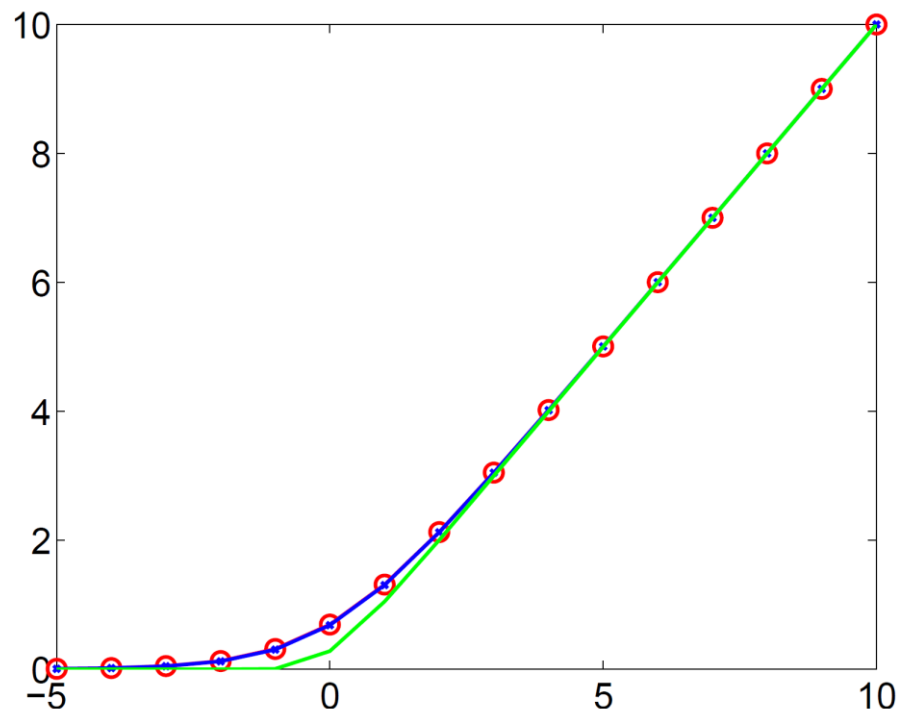


$$f(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

激活函数—ReLU (Rectified Linear Units)

- 受限玻尔兹曼机

$$\sum_{i=1}^N \sigma(x - i + 0.5) \approx \log(1 + e^x)$$

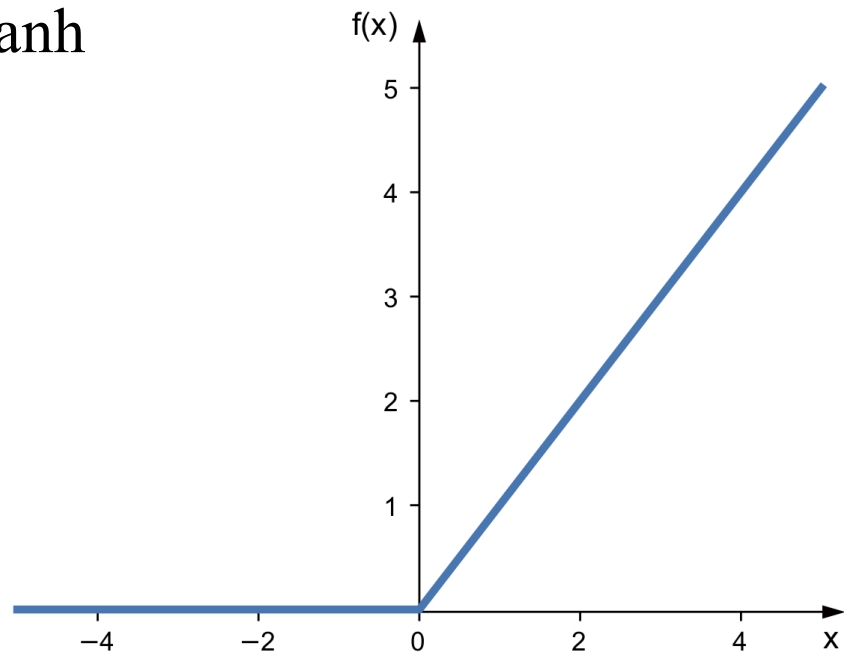
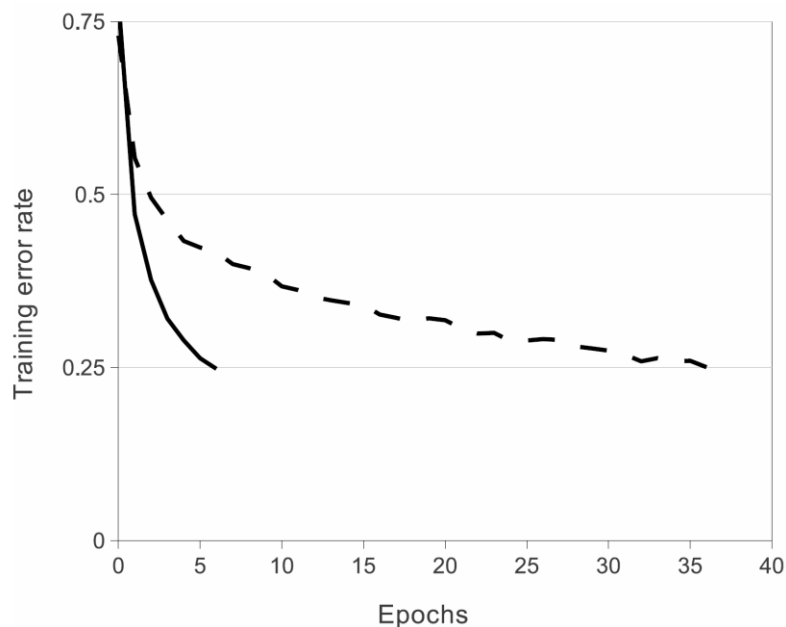


Vinod Nair, Geoffrey E. Hinton, Rectified Linear Units Improve Restricted Boltzmann Machines, ICML 2010.

激活函数—ReLU

- AlexNet $f(x) = \max(0, x)$

solid line: ReLU; dashed line: tanh

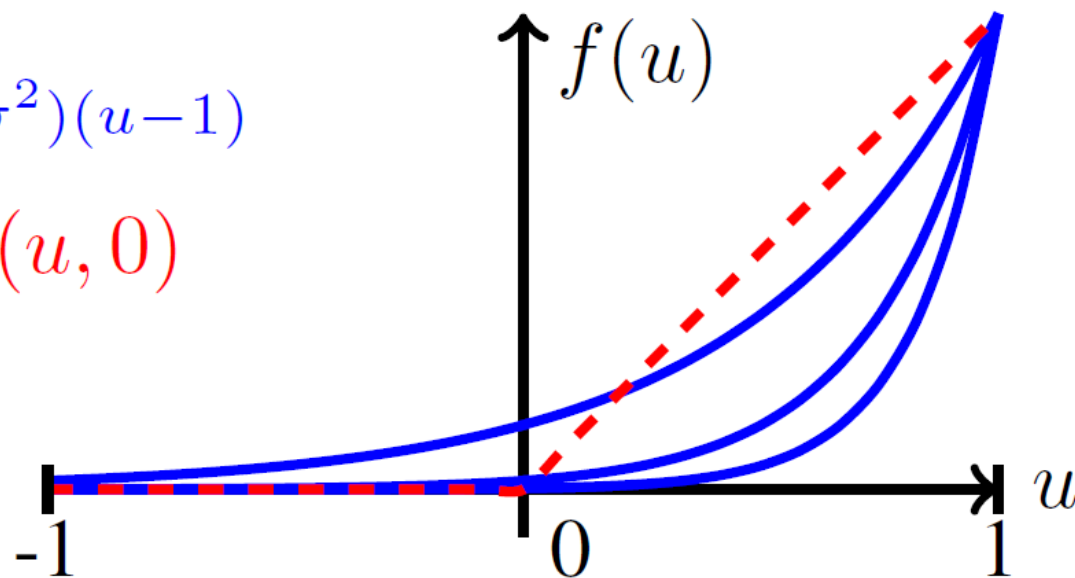


A Krizhevsky, I Sutskever, GE Hinton, Imagenet classification with deep convolutional neural networks, NIPS 2012

激活函数—ReLU

$$f(u) = e^{(2/\sigma^2)(u-1)}$$

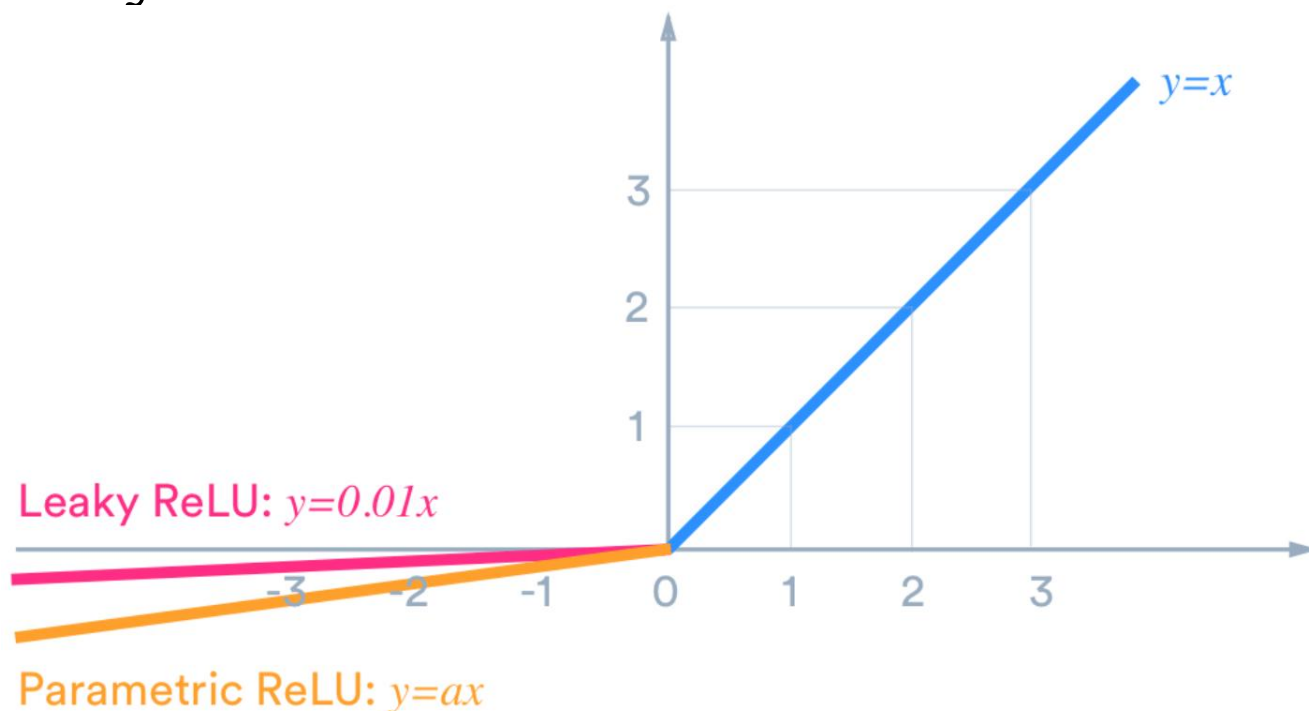
$$f(u) = \max(u, 0)$$



J Mairal, P Koniusz, Z Harchaoui, C Schmid, Convolutional Kernel Networks, NIPS 2014

ReLU变体及改进

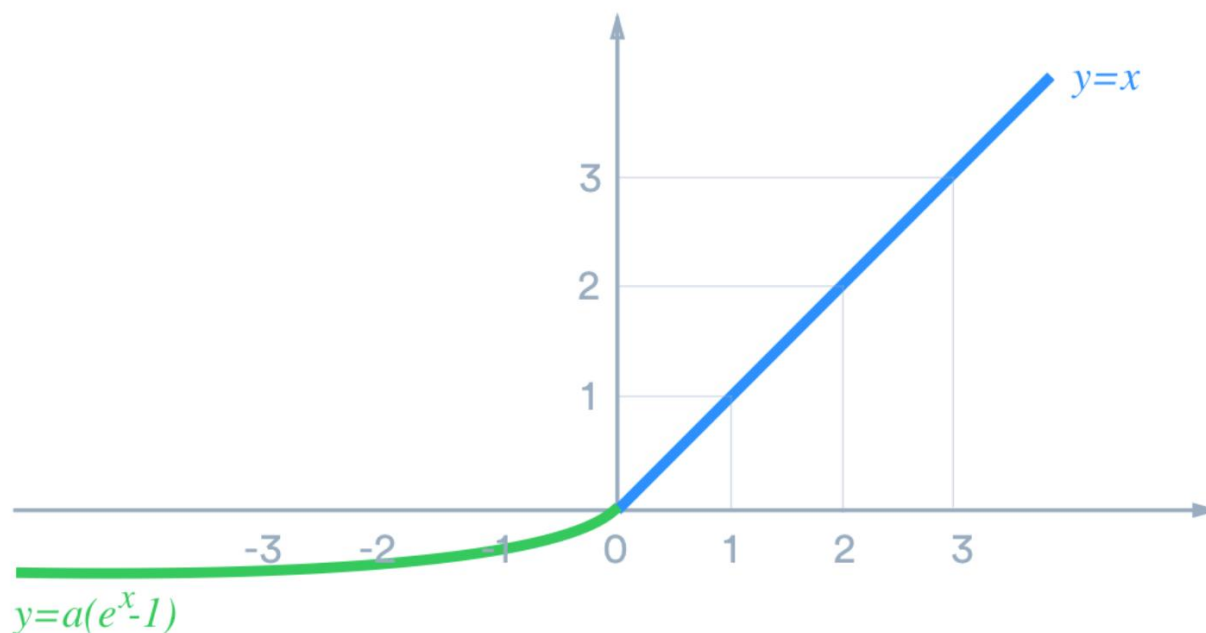
- Leaky ReLU & PReLU



K He, X Zhang, S Ren, J Sun, Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification, ICCV 2015

ReLU变体及改进

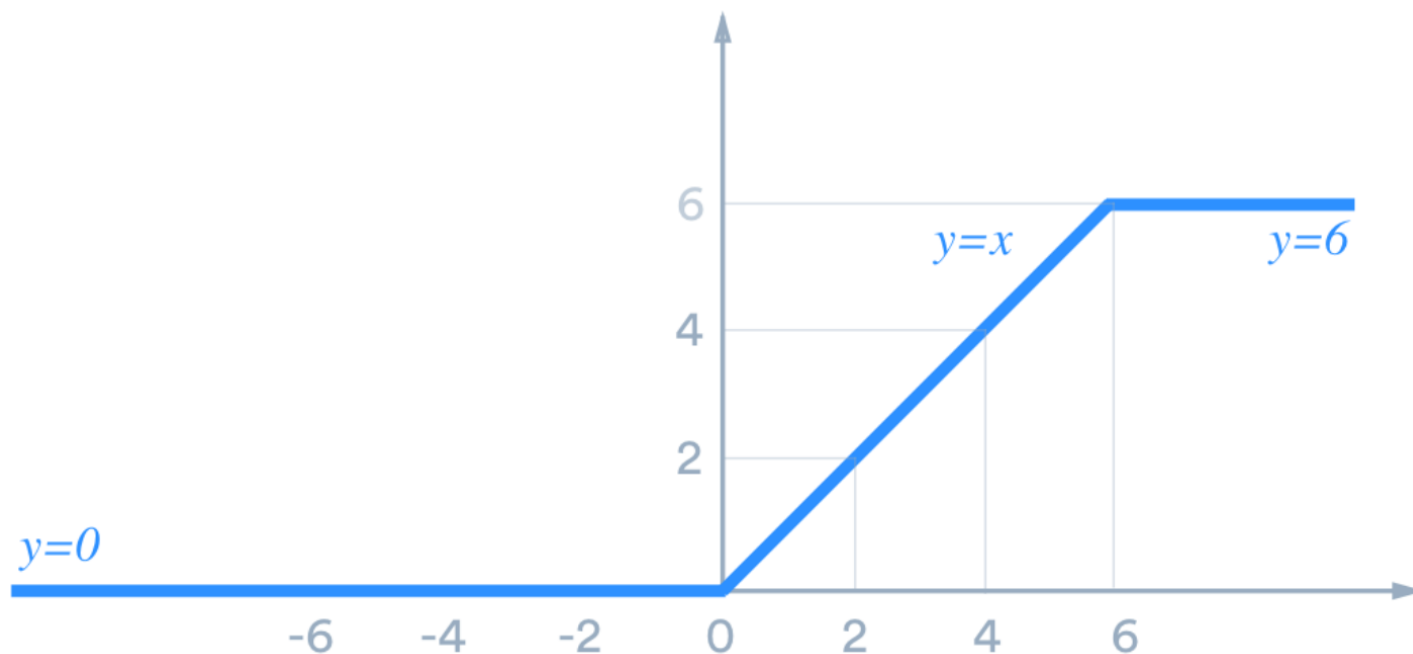
- ELU: Exponential Linear Units



D.-A. Clevert, T. Unterthiner, S. Hochreiter, Fast and Accurate Deep Network Learning by Exponential Linear Units (ELUs), ICLR 2016

ReLU变体及改进

- ReLU-6

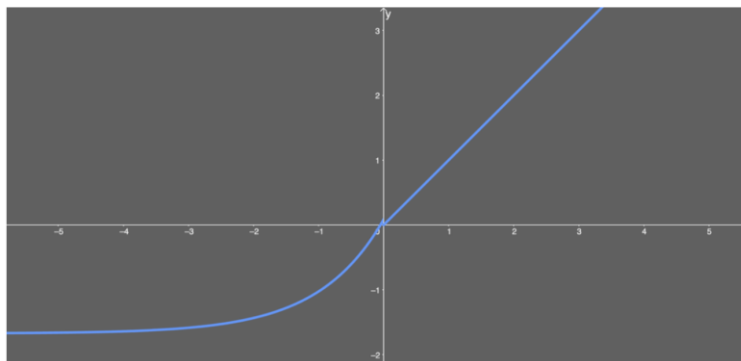


Alex Krizhevsky, Convolutional Deep Belief Networks on CIFAR-10, Technic Report, 2010

其它激活函数

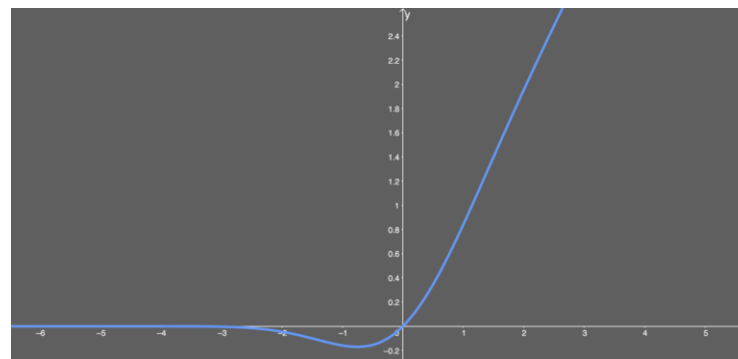
- SELU

$$\text{SELU}(x) = \lambda \begin{cases} x & \text{if } x > 0 \\ \alpha e^x - \alpha & \text{if } x \leq 0 \end{cases}$$



- GELU

$$\text{GELU}(x) = 0.5x \left(1 + \tanh \left(\sqrt{2/\pi} (x + 0.044715x^3) \right) \right)$$



- 从ReLU到GELU，一文概览神经网络的激活函数：
<https://mp.weixin.qq.com/s/pA9JW75p9J5e5KHe3ifcBQ>

Revisit GELU

- Revisit GELU

$$GELU(x) = x\Phi(x)$$

$$0.5x(1 + \tanh[\sqrt{2/\pi}(x + 0.044715x^3)])$$

- SimpleGate

$$Gate(\mathbf{X}) = f(\mathbf{X}) \odot g(\mathbf{X})$$

	GELU→SG	CA→SCA	SIDD		GoPro	
			PSNR	SSIM	PSNR	SSIM
Baseline	✓	✓	39.85	0.959	32.35	0.956
			39.93	0.960	32.76	0.960
			39.95	0.960	32.54	0.958
NAFNet	✓	✓	39.96	0.960	32.85	0.960

主要内容

- 随机优化算法（Stochastic Optimization）
 - 随机优化
 - SGD、AdaGrad、RMSProp、ADAM
- 网络模型：3层 -> 多层
 - 激活函数：ReLU / PReLU, ...
 - 网络参数：**Dropout**、批标准化（Batch Normalization）
- 泛化能力

模型平均与Bagging

- 模型平均（集成学习）

假设我们有 k 个回归模型。假设每个模型在每个例子上的误差是 ϵ_i ，这个误差服从零均值方差为 $\mathbb{E}[\epsilon_i^2] = v$ 且协方差为 $\mathbb{E}[\epsilon_i \epsilon_j] = c$ 的多维正态分布。通过所有集成模型的平均预测所得误差是 $\frac{1}{k} \sum_i \epsilon_i$ 。集成预测器平方误差的期望是

$$\mathbb{E} \left[\left(\frac{1}{k} \sum_i \epsilon_i \right)^2 \right] = \frac{1}{k^2} \mathbb{E} \left[\sum_i \left(\epsilon_i^2 + \sum_{j \neq i} \epsilon_i \epsilon_j \right) \right], \quad (7.50)$$

$$= \frac{1}{k} v + \frac{k-1}{k} c. \quad (7.51)$$

在误差完全相关即 $c = v$ 的情况下，均方误差减少到 v ，所以模型平均没有任何帮助。在错误完全无关即 $c = 0$ 的情况下，该集成平方误差的期望仅为 $\frac{1}{k} v$ 。

模型平均与Bagging

- Bagging

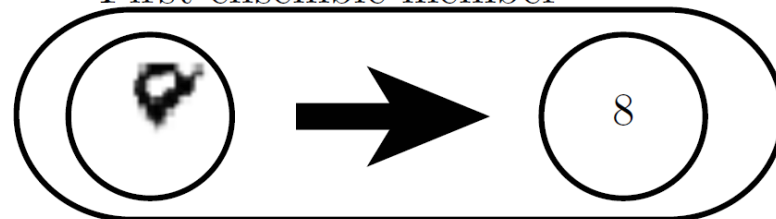
Original dataset



First resampled dataset



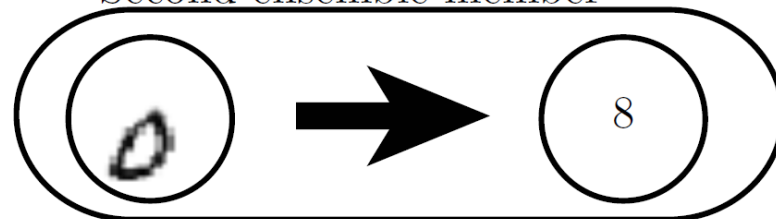
First ensemble member



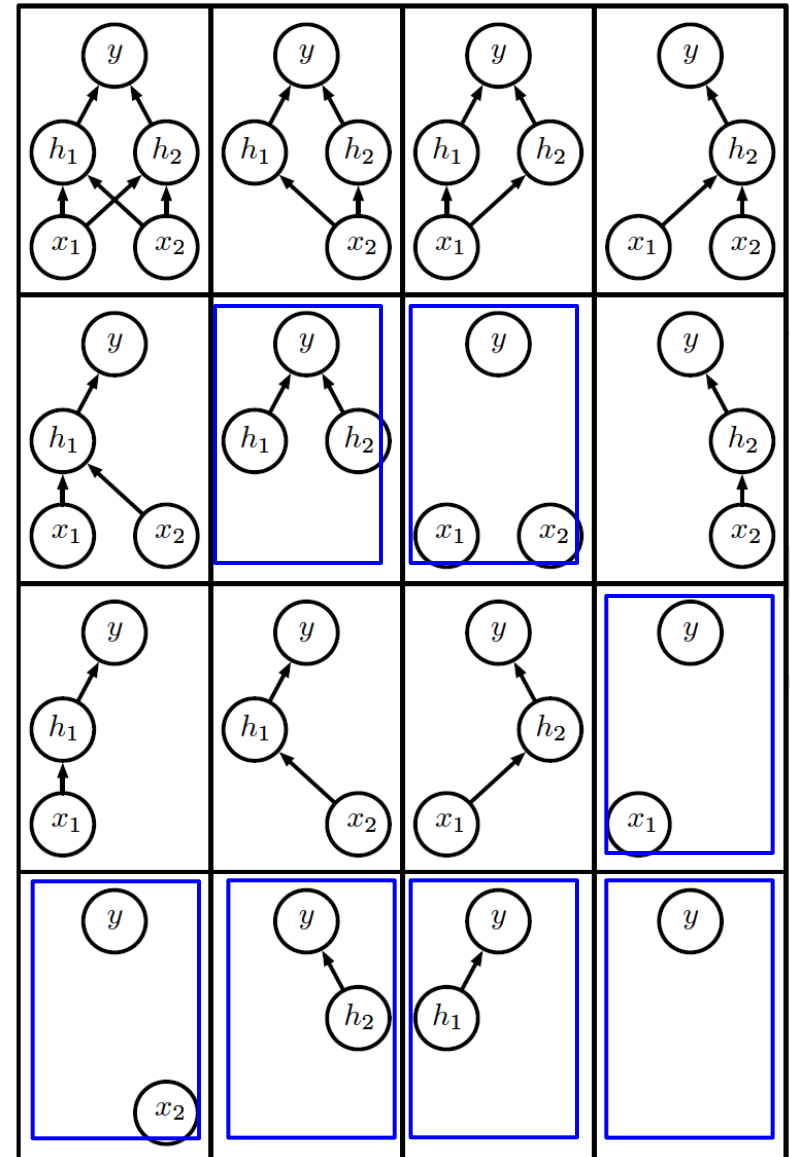
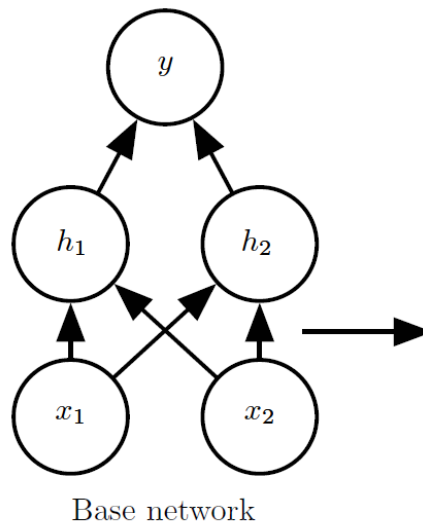
Second resampled dataset



Second ensemble member

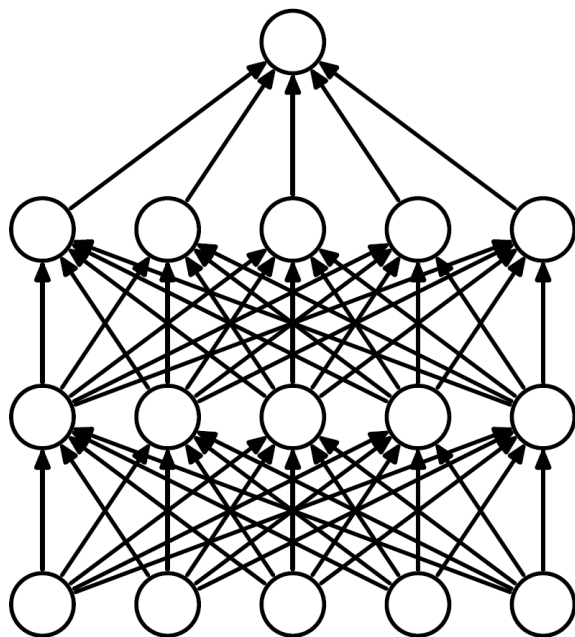


Dropout

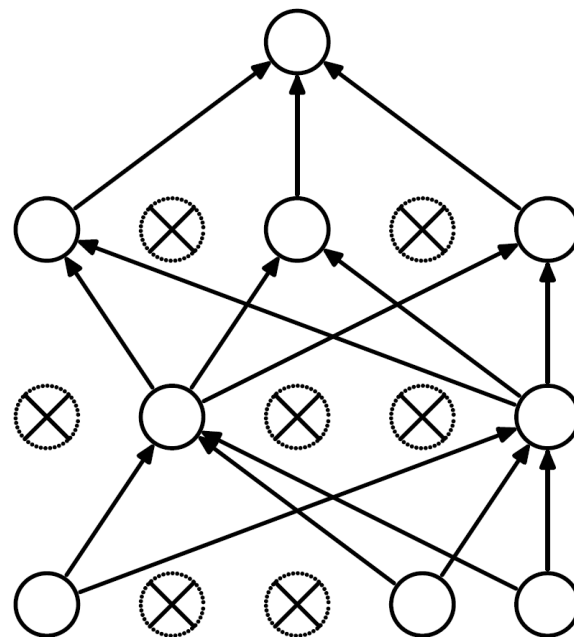


Ensemble of subnetworks

Dropout训练



(a) Standard Neural Net



(b) After applying dropout.

- 在小批量中加载一个样本，然后随机抽样应用于网络中所有输入和隐藏单元的不同二值掩码
- 超参数：掩码值为1 的采样概率

Dropout训练

$$\begin{aligned} z_i^{(l+1)} &= \mathbf{w}_i^{(l+1)} \mathbf{y}^l + b_i^{(l+1)} \\ y_i^{(l+1)} &= f(z_i^{(l+1)}) \end{aligned}$$

Dropout 

$$\begin{aligned} r_j^{(l)} &\sim \text{Bernoulli}(p) \\ \tilde{\mathbf{y}}^{(l)} &= \mathbf{r}^{(l)} * \mathbf{y}^{(l)} \\ z_i^{(l+1)} &= \mathbf{w}_i^{(l+1)} \mathbf{y}^l + b_i^{(l+1)} \\ y_i^{(l+1)} &= f(z_i^{(l+1)}) \end{aligned}$$

Dropout

- 训练阶段所有模型共享参数，测试阶段直接组装成一个整体的大网络
- 有效避免过拟合
- 可用于前馈神经网络、概率模型，如受限玻尔兹曼机，以及循环神经网络等
- 会需要较多的迭代次数和训练时间
- 理论解释



理论解释

- First-order equivalent to an L2 regularizer applied after scaling the features by an estimate of the inverse diagonal Fisher information matrix.
 - Understanding Dropout, NIPS 2013
 - Dropout Training as Adaptive Regularization, NIPS 2013
- 

主要内容

- 随机优化算法（Stochastic Optimization）
 - 随机优化
 - SGD、AdaGrad、RMSProp、ADAM
- 网络模型：3层 -> 多层
 - 激活函数：ReLU / PReLU, ...
 - 网络参数：Dropout、批标准化（Batch Normalization）
- 泛化能力



批标准化（Batch Normalization）

- Bengio: 最激动人心的进展之一，自适应重参数化，减少了多层之间协调更新，有助于训练非常深的模型。
 - Mini-batch SGD: Internal covariate shift
 - 改善网络学习稳定性
 - 正则化
- 

批标准化 (Batch Normalization)

- $z = g(\text{BN}(Wu))$

- 测试阶段:
- 用训练阶段收集的运行均值和方法代替Batch均值和方差

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1...m}\}$;

Parameters to be learned: γ, β

Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$

$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scale and shift}$$

反向传播

$$\frac{\partial \ell}{\partial \hat{x}_i} = \frac{\partial \ell}{\partial y_i} \cdot \gamma$$

$$\frac{\partial \ell}{\partial \sigma_{\mathcal{B}}^2} = \sum_{i=1}^m \frac{\partial \ell}{\partial \hat{x}_i} \cdot (x_i - \mu_{\mathcal{B}}) \cdot \frac{-1}{2} (\sigma_{\mathcal{B}}^2 + \epsilon)^{-3/2}$$

$$\frac{\partial \ell}{\partial \mu_{\mathcal{B}}} = \left(\sum_{i=1}^m \frac{\partial \ell}{\partial \hat{x}_i} \cdot \frac{-1}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \right) + \frac{\partial \ell}{\partial \sigma_{\mathcal{B}}^2} \cdot \frac{\sum_{i=1}^m -2(x_i - \mu_{\mathcal{B}})}{m}$$

$$\frac{\partial \ell}{\partial x_i} = \frac{\partial \ell}{\partial \hat{x}_i} \cdot \frac{1}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} + \frac{\partial \ell}{\partial \sigma_{\mathcal{B}}^2} \cdot \frac{2(x_i - \mu_{\mathcal{B}})}{m} + \frac{\partial \ell}{\partial \mu_{\mathcal{B}}} \cdot \frac{1}{m}$$

$$\frac{\partial \ell}{\partial \gamma} = \sum_{i=1}^m \frac{\partial \ell}{\partial y_i} \cdot \hat{x}_i$$

$$\frac{\partial \ell}{\partial \beta} = \sum_{i=1}^m \frac{\partial \ell}{\partial y_i}$$

批标准化（Batch Normalization）

- 优点：
 - 正则化
 - 允许设置较高的学习率
 - Internal covariate shift
 - 可代替dropout
 - 可代替L2正则化
- 会需要较大的batch size。一般会有效，但不能保证任何情况下都会有效

Going beyond (1)

- Batch Normalization on Feature Maps
- Orthogonal regularization on convolution filters

$$\|W^T W - I\|_F^2$$

S. Ioffe and C. Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift.

In *ICML 2015, Lille, France, 6-11 July 2015*, pages 448–456, 2015.

K. Jia, D. Tao, S. Gao, and X. Xu, Improving training of deep neural networks via Singular Value Bounding, CVPR 2017

Going beyond (2)

- Weight normalization

$$\sigma_1(\bar{W}_{\text{WN}})^2 + \sigma_2(\bar{W}_{\text{WN}})^2 + \cdots + \sigma_T(\bar{W}_{\text{WN}})^2 = d_o$$

- Spectral normalization

$$\bar{W}_{\text{SN}}(W) := W/\sigma(W)$$

T. Salimans and D.P. Kingma. Weight normalization: A simple reparameterization to accelerate training of deep neural networks. In NIPS, pp. 901–909, 2016.

T. Miyato, T. Kataoka, M. Koyama, Y. Yoshida, Spectral normalization for generative adversarial networks, ICLR 2018

主要内容

- 随机优化算法（Stochastic Optimization）
 - 随机优化
 - SGD、AdaGrad、RMSProp、ADAM
- 网络模型：3层 -> 多层
 - 激活函数：ReLU / PReLU, ...
 - 网络参数：Dropout、批标准化（Batch Normalization）
- 泛化能力

反思

- 3层 -> 多层
- SGD: 使用SGD原因的变迁
- SGD的改进: (几何解释、直觉) $\leftrightarrow$ 理论分析
- Sigmoid -> ReLU: 敢于突破前人的认知
- Dropout: 大模型 $\leftrightarrow$ 小模型
- Batch Normalization: 特征和权重视角